

THE CRUX: HOW TO SPEED UP ADDRESS TRANSLATION

How can we speed up address translation, and generally avoid the extra memory reference that paging seems to require? What hardware support is required? What OS involvement is needed?

To speed-up address translation, we are going to add a translation-lookaside buffer (TLB) [A cache of sorts]

A TLB is part of memory-management unit.

On each virtual memory access, the hardware checks TLB to see if the desired translation is held therein.

If so, translation is performed quickly without having to consult page table.

TLB Basic Algorithm

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else    // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19     RetryInstruction()
```

Figure 19.1: TLB Control Flow Algorithm

Assuming linear page table & a hardware managed TLB

The algorithm the hardware follows:

1. Extract VPN from virtual address
2. Check if TLB holds PFN for this VPN
3. If yes,
we have a TLB hit.
4. Assuming protection bit do not fail
- 5-7. we can extract PFN, concatenate that onto offset from virtual address, form desired Physical address & access memory.
10. If no,
we have a TLB miss
- 11-12. The hardware accesses the page table to find the translation i.e PTE
- 13-14. Assume that virtual address is valid & accessible
18. Update TLB with translation i.e PTE
19. Retry the instruction

Example,

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

In this example, we have a 40 byte array starting at virtual address 100.

Assume 8-bit address space with 16-byte pages. So, virtual address splits as:

1. 4 bit for VPN

2. 4 bit offset

	Offset				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

↳ The array spans 3 pages.

Virtual address
space

For simplicity, we assume that the only memory addresses the loop generates are to the array.

When $a[0]$ is accessed, CPU loads VA 100. The hardware extracts $VPN = 06$ & uses that to check the TLB for valid translation.

Assuming that this will be the first program access, the result will be TLB miss.

When $a[1]$ is accessed, as hardware extracts $VPN = 06$ we get a TLB hit, same for $a[2]$.

When $a[3]$ is accessed, we get TLB miss but for $a[4] \dots a[6]$ we get TLB hit.

when $a[7]$ is accessed, we get TLB miss but for $a[8], a[9]$ we get TLB hit.

So, our hit rate is:

$$\frac{7}{10} \times 100 = 70\%$$

↳ Note that if page size was bigger (32 bytes not 16), then

would be even fewer misses.

Who handles TLB miss?

- The hardware in x86, CISC:

- ↳ To do this, hardware needs to know where page table is in memory via a page table base register.

- ↳ Their extract format

- ↳ On miss, hardware would find correct page table entry & extract the desired translation, update the TLB with the translation, & retry the instruction.

- The OS in RISC:

- ↳ On TLB miss, the hardware raises an exception, which pauses the current instruction stream, raises privilege level to kernel mode, jump to trap handler.

- ↳ Within the trap handler, the code will look-up translation, use special privileged instructions to update the TLB, return from trap, & hardware retries the instruction.

This return-from-trap is different from when servicing a system call.

Before return-from-trap resumed execution at the instruction after the one that caused trap into OS.

In this case, return-from-trap should resume instruction that caused the trap i.e. retry the instruction.

Thus, depending on how a trap or exception was caused, the hardware must save a different PC when trapping into the OS, in order to resume properly when the time to do so arrives.

Second, when running the TLB miss-handling code, the OS needs to be extra careful not to cause an infinite chain of TLB misses to occur. Many solutions exist; for example, you could keep TLB miss handlers in physical memory (where they are **unmapped** and not subject to address translation), or reserve some entries in the TLB for permanently-valid translations and use some of those permanent translation slots for the handler code itself; these **wired** translations always hit in the TLB.

Advantage of this:

1. Flexibility — OS can use any data structure for page table
2. Simplicity — the hardware doesn't do much. So, simple circuit.

ASIDE: TLB VALID BIT $\neq$ PAGE TABLE VALID BIT

A common mistake is to confuse the valid bits found in a TLB with those found in a page table. In a page table, when a page-table entry (PTE) is marked invalid, it means that the page has not been allocated by the process, and should not be accessed by a correctly-working program. The usual response when an invalid page is accessed is to trap to the OS, which will respond by killing the process.

A TLB valid bit, in contrast, simply refers to whether a TLB entry has a valid translation within it. When a system boots, for example, a common initial state for each TLB entry is to be set to invalid, because no address translations are yet cached there. Once virtual memory is enabled, and once programs start running and accessing their virtual address spaces, the TLB is slowly populated, and thus valid entries soon fill the TLB.

The TLB valid bit is quite useful when performing a context switch too, as we'll discuss further below. By setting all TLB entries to invalid, the system can ensure that the about-to-be-run process does not accidentally use a virtual-to-physical translation from a previous process.

TLB contents: What's in there?

↳ A typical TLB has 32, 64 or 128 entries & called fully associative.

Given an translation can be anywhere in the TLB, and that the hardware will search the entire TLB in parallel to find the desired translation.

A TLB entry might look like

VPN		PFN		other bits
-----	--	-----	--	------------

Valid bit, Protection bits, address-space identifiers, dirty bit.

TLB Issue: Context switch

TLB contains virtual-to-physical translations that are not meaningful for other processes.

As a result, when switching from one process to another, the hardware must ensure that

Example,

When P1 is running, it assumes TLB might be caching translations valid for it i.e. that came from P1's page table.

Assume for P1, VPN 10 $\rightarrow$ PFN 100

When OS decides to run P2, assume also that it's VPN 10 $\rightarrow$ PFN 170.

So, our TLB is:

VPN	PFN	valid	prot
10	100	1	rwX
—	—	0	—
10	170	1	rwX
—	—	0	—

Now, VPN 10 translates to PFN 100 or PFN 170 but the hardware doesn't distinguish

THE CRUX:

HOW TO MANAGE TLB CONTENTS ON A CONTEXT SWITCH

When context-switching between processes, the translations in the TLB for the last process are not meaningful to the about-to-be-run process. What should the hardware or OS do in order to solve this problem?

Flush:

Empty TLB before running the next process

On a software-based system, this can be accomplished with an explicit (and privileged)

hardware instruction; with a hardware-managed TLB, the flush could be enacted when the page-table base register is changed (note the OS must change the PTBR on a context switch anyhow). In either case, the flush operation simply sets all valid bits to 0, essentially clearing the contents of the TLB.

Now, each time a process runs, it must incur TLB misses. If OS switches frequently, the cost may be high.

Address Space Identifier(ASID)

↳ In some systems, the hardware supports sharing of TLB across context switching through ASID.

Now, our above TLB looks like:

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
—	—	0	—	—
10	170	1	rwX	2
—	—	0	—	—

Hardware needs to know which process is running, thus OS on context switch set a privileged register to ASID of the process.

Two processes might share a page (wdc) that point to same physical page.

VPN	PFN	valid	prot	ASID
10	101	1	r-x	1
—	—	0	—	—
50	101	1	r-x	2
—	—	0	—	—

A real TLB entry:

MIPS R4000 supports a 32 bit address space with 4KB pages. So, we would have 20 bit VPN & 12 bit offset.

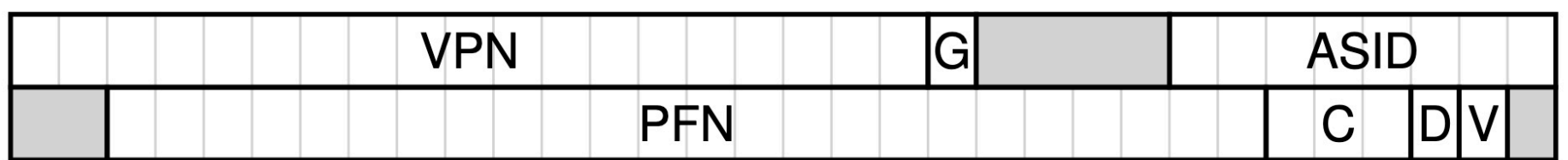


Figure 19.4: A MIPS TLB Entry

but there is (fig 19.4) 19 VPN bits as user addresses will only come from half the address space, the rest is reserved for kernel.

The VPN translates upto 24-bit physical frame number & hence support upto 64GB RAM (2^{24} 4KB pages)

Global bit (G):

↳ Used for pages shared that are globally shared among processes.

Thus, if G is 1, ASID is ignored.

Coherence bit (C):

↳ Determines how a page is cached by hardware

A dirty bit (D):

↳ Which is marked when a page has been written to.

Page mask field:

Supports multiple page sizes.

MIPS TLB has 32 or 64 of these entries.

Few entries are reserved for the OS. A wired register set by OS tells hardware how many slots of TLB to reserve for the OS.

These mappings are used by OS when TLB miss will be problematic for instance, TLB miss handler.

As MIPS TLB is software managed. MIPS provides four instructions to update TLB.

TLBP, which probes the TLB to see if a particular translation is in there; TLBR, which reads the contents of a TLB entry into registers; TLBWI, which replaces a specific TLB entry; and TLBWR, which replaces a random TLB entry. The OS uses these instructions to manage the TLB's contents. It is of course critical that these instructions are **privileged**;

Disadvantages

If the number of pages a program accesses in a short period of time exceeds the number of pages that fit into the TLB, the program will generate a large number of TLB misses, and thus run quite a bit more slowly. We refer to this phenomenon as exceeding the TLB coverage, and it can be quite a problem for certain programs.

TLB access can easily become a bottleneck in the CPU pipeline, in particular with what is called a physically-indexed cache. With such a cache, address translation has to take place before the cache is accessed, which can slow things down quite a bit

→ The cache is organized based on physical addresses.